

Teaching Burp a new HTTP Transport Encoding

Teaching Burp a new HTTP Transport Encoding - Pentagrid AG, @pentagridsec, Pentagrid AG.

- Published: 2022-04-13
- Original:
<https://www.pentagrid.ch/en/blog/teaching_burp_a_new_http_transport_encoding/>
- Preserved from:
https://www.pentagrid.ch/en/blog/teaching_burp_a_new_http_transport_encoding/ (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Today we would like to talk about Burp Suite Professional and extensions again. In this blog post we explain how we can teach Burp Suite to handle a custom Transport Encoding that is spoken between an HTTP client and a server by using Burp extensions.

Introduction

Burp has many nice features, but sometimes it lacks a feature we would like to have or our target application is customized so that it is necessary to teach Burp new things. More specifically:

-

Burp has [no support for the standard HTTP brotli Transport-Encoding](#), but all modern browsers do. This problem is handled by Burp by removing brotli from the Accept-Encoding request header (see Proxy - Options - Miscellaneous - Remove unsupported encodings from Accept-Encoding headers in incoming requests), which works for modern browsers. However, we encountered situations during tests where a non-browser HTTP client software required brotli and refused to talk to the server when brotli encoding is missing.

-

Java fat clients sometimes zlib-encode the entire HTTP body when talking to a server and vice-versa. They also often refuse to speak non-zlib and do not signal it at all in an HTTP

header, therefore it is necessary to implement zlib-decoding and encoding to be able to use Burp properly.

-

If a website uses JavaScript to encrypt all HTTP bodies when talking to a server and vice-versa, we also would like to implement a decryption and encryption routing to use the Burp tools. We also encountered the same situation with other HTTP clients that used additional transport encryption, such as mobile banking apps.

The biggest issue for us was to figure out how we approach these situations consistently and we show our approach in this blog post.

In this blog post we often refer to "encoding" or "encrypting", which obviously is not the same, but both operations have to be done at the same place inside Burp. Encryption or encoding is merely an implementation detail in this blog post. Also "cleartext" is used to indicate decoded or decrypted content.

Why are you doing all of this?

If Burp is not seeing decoded messages for one of the reasons above, it won't work properly. Tools such as Burp Scanner, Repeater, Intruder and other extensions will often fail to interpret the encoded HTTP content and will for sure fail with encrypted content. It won't matter if you use the scanner or not, in many cases it won't do anything useful if you don't implement this custom decoder/encoder. On the contrary, very often a tester's task is to make their tools work. This might be rewarded in the end by simply pressing the active scan button in Burp and then finding security issues. To us, this is a huge difference between a good and a bad security analysis.

Usually we would suggest to solve this problem with features or extensions that are already present, for example the Hackvertor extension would be a good candidate. However, Hackvertor won't modify responses, so this isn't an option here.

Burp's API for extensions

Burp provides several [APIs an extension can use](#). The API code didn't get much love in the last few years, but this is hopefully changing when Burp releases the [New API and multi-language extensibility](#). Let's hope for the best, but for now we're stuck with the API we have. There are two main API hooks that matter in the situation of a missing Transport Encoding. We'll use the name of the function name in the rest of this blog post, but the interfaces that provide the functionality are called:

-

[IProxyListener](#), which means you have to implement the function `processProxyMessage`.

-

[IHttpListener](#), which means you have to implement the function `processHttpRequest`.

While both interfaces sound similar, the proxy listener will only see requests and responses going through the proxy (e.g. from the browser), whereas the HTTP listener will see the requests and responses of all tools (Proxy, Crawler, Scanner, Repeater, Intruder, other extensions, etc.). However, there's one little detail that is important to mention: If you want to change messages going through the proxy, you have to do that in the `processProxyMessage` function and you can't do that in the `processHttpMessage`. It's just a Burp API limitation that you can't modify messages from `TOOL_PROXY` in the `processHttpMessage`. That means we have to get around this limitation by choosing the correct API.

While we think it is better to use Kotlin to write large Burp extensions nowadays, rapid prototyping is still quicker with Python. Therefore we use Python (or rather, Jython 2.7 in Burp) in this blog post.

Here's a little Python sample extension that handles the API confusion of `processProxyMessage` and `processHttpMessage`, so we don't need to care about it anymore and can just modify messages in the newly created `processMessage` function:

```

from burp import IBurpExtender
from burp import IHttpListener
from burp import IProxyListener
from burp import IBurpExtenderCallbacks

NAME = "Pentagrid Extension Template"

class BurpExtender(IBurpExtender, IProxyListener, IHttpListener):

    def registerExtenderCallbacks(self, callbacks):
        # keep a reference to our callbacks object
        self._callbacks = callbacks

        # set our extension name
        callbacks.setExtensionName(NAME)

        # register ourselves as an Proxy listener
        callbacks.registerProxyListener(self)

        # register ourselves as an HTTP listener
        callbacks.registerHttpListener(self)

        print("Loaded "+NAME+" successfully!")

    #
    # implement IHttpListener
    #

    def processHttpMessage(self, toolFlag, messageIsRequest, messageInfo):
        if toolFlag == IBurpExtenderCallbacks.TOOL_PROXY:
            # DONT'T DO ANYTHING HERE
            # Instead, use processProxyMessage below
            # https://github.com/nccgroup/BurpSuiteLoggerPlusPlus/issues/42
            return
        self.processMessage(toolFlag, messageIsRequest, messageInfo)

    #
    # implement IProxyListener
    #

    def processProxyMessage(self, messageIsRequest, message):
        self.processMessage(IBurpExtenderCallbacks.TOOL_PROXY, messageIsRequest,
message.getMessageInfo())

    def processMessage(self, toolFlag, messageIsRequest, messageInfo):

```

```
# TODO: Implement whatever you would like to do
pass
```

As a bonus, the Burp UI will show the "Edited request" or "Edited response" option in the Proxy tool whenever we change a proxy message. However, for `processHttpRequest` we are partially blind, the request and responses will simply be changed, but depending on where we measure we might see them in a different state in the UI (discussed in the next section).

Burp's hooking locations

We would like to hook as early as possible and as late as possible. We would like to make sure that we decode or decrypt early, so that all other Burp tools can work with readable non-compressed HTTP requests and responses. We also want to encode or encrypt as late as possible, just before the request leaves Burp for the same reason. The problem here is that `processProxyMessage` is called before `processHttpRequest` for requests, but for responses `processHttpRequest` is called before `processProxyMessage`. When analysing the exact behavior, we saw it's a non-trivial setup. For example, the built-in Burp Logger is a tab in the Burp UI showing all requests/responses that flow through Burp. But the question is, where does the Burp Logger hook the API itself? Moreover, users who have advanced knowledge of Burp also know that the extension order in the loaded extension list matters. So where should we put our decoder or decrypter and where should we put our encoder or encrypter? This is not easy to explain and it took us a little trial and error, but we figured out that this diagram should be accurate:

[[image: Diagram showing how requests and responses are processed inside Burp when we load our Burp extensions. Green is for cleartext \(decoded or decrypted\) traffic, red for encoded or encrypted traffic.](#)]

(https://www.pentagrid.ch/images/202204_burp_handling_of_messages.png)

Diagram showing how requests and responses are processed inside Burp when we load our Burp extensions. Green is for cleartext (decoded or decrypted) traffic, red for encoded or encrypted traffic.

This diagram shows several interesting details:

-

We achieve what we want: Target, Intruder, Repeater, Scanner and other extensions can deal with "cleartext" requests. However, we can't make it work with Proxy match/replace rules for requests (they are applied to the encoded content).

-

Burp's built-in Logger is not last in the chain for responses, it's possible that Burp extensions modify the responses. This means you don't necessarily see what was delivered to the client/server in Logger. You should better use Logger++ and put it last in the extension list to see which requests are sent to the server and which responses are

returned after processing in Burp. Of course it depends on what you want to see, but that's what is the most logical setup.

-

Our extension we would like to write for our purpose has to be first *and* last (well, second-last before Logger++) to make sure it does its job well. This means we need to write two extensions, an early-decoder that is first in the list for requests (see DECRYPTER top left in the diagram) and responses (see DECRYPTER bottom right) and a late-reencoder that is last in the list for requests (see ENCRYPTER bottom left) and responses (see ENCRYPTER top right). So the order of the extensions has to be like in the following image:

[[image: Extension order is important to make sure all extensions see the correct traffic. Early decoder as the first extension, late encoder as the second-last.]]

(https://www.pentagrid.ch/images/202204_burp_extension_order.png)

Extension order is important to make sure all extensions see the correct traffic. Early decoder as the first extension, late encoder as the second-last.

Now that we know we need two extensions, how would the Python extensions look like? Here's the early decoder:

```

from burp import IBurpExtender
from burp import IHttpListener
from burp import IProxyListener
from burp import IBurpExtenderCallbacks

NAME="Pentagrid early decoder"

class BurpExtender(IBurpExtender, IHttpListener, IProxyListener):

    def registerExtenderCallbacks(self, callbacks):

        # keep a reference to our callbacks object
        self._callbacks = callbacks

        # obtain an extension helpers object
        self._helpers = callbacks.getHelpers()

        # set our extension name
        callbacks.setExtensionName(NAME)

        # register ourselves as an HTTP listener
        callbacks.registerHttpListener(self)
        callbacks.registerProxyListener(self)

        print("Loaded "+NAME+" successfully!")

    #
    # implement IHttpListener
    #

    def processHttpMessage(self, toolFlag, messageIsRequest, messageInfo):
        if toolFlag == IBurpExtenderCallbacks.TOOL_PROXY and messageIsRequest:
            # Already processed in processProxyMessage
            return
        self.filter_message(toolFlag, messageIsRequest, messageInfo)

    #
    # implement IProxyListener
    #

    def processProxyMessage(self, messageIsRequest, message):
        # Responses are handled as early as possible in processHttpMessage
        if messageIsRequest:
            self.filter_message(IBurpExtenderCallbacks.TOOL_PROXY, messageIsRequest,
message.getMessageInfo())

```

```
def filter_message(self, toolFlag, messageIsRequest, messageInfo):  
    # TODO: implement decoding  
    pass
```

And here's the late encoder:

```

from burp import IBurpExtender
from burp import IHttpListener
from burp import IProxyListener
from burp import IBurpExtenderCallbacks

NAME="Pentagrid late encoder"

class BurpExtender(IBurpExtender, IHttpListener, IProxyListener):

    def registerExtenderCallbacks(self, callbacks):

        # keep a reference to our callbacks object
        self._callbacks = callbacks

        # obtain an extension helpers object
        self._helpers = callbacks.getHelpers()

        # set our extension name
        callbacks.setExtensionName(NAME)

        # register ourselves as an HTTP listener
        callbacks.registerHttpListener(self)
        callbacks.registerProxyListener(self)

        print("Loaded "+NAME+" successfully!")

    #
    # implement IHttpListener
    #

    def processHttpMessage(self, toolFlag, messageIsRequest, messageInfo):
        if toolFlag == IBurpExtenderCallbacks.TOOL_PROXY and not messageIsRequest:
            # Requests are handled as late as possible in processProxyMessage
            return
        self.filter_message(toolFlag, messageIsRequest, messageInfo)

    #
    # implement IProxyListener
    #

    def processProxyMessage(self, messageIsRequest, message):
        # Responses are handled as late as possible in processHttpMessage
        if not messageIsRequest:
            self.filter_message(IBurpExtenderCallbacks.TOOL_PROXY, messageIsRequest,
message.getMessageInfo())

```

```
def filter_message(self, toolFlag, messageIsRequest, messageInfo):  
    # TODO: implement encoding  
    pass
```

Bonus

To make this a little more convenient for you, we've set up a [Github repository](#) with all the examples from above (in the "minimal" subfolder). However, that's not all:

-

A fully working example for the zlib-body encoding/decoding.

-

A template that has some more helpful functions if you want to implement your own decoding/encoding.

-

An additional extension that implements an IMessageEditorTabFactory, so if you encounter an encoded message in Burp you can click the message viewer option of the extension and will see it in decoded form.

More advanced examples

With the above examples it should be no problem for you to also create extensions that do decryption/encryption instead of decoding/encoding in more complicated setups. In one of our security analysis, we were able to implement the decryption/encryption and hardcoded the key at first. However, then we realised we could additionally inject a JavaScript tag into the HTML single-page response of the server to "steal" our own encryption key from our browser. This removed the requirement to hard-code the encryption key. We did this by replacing `</body>` with:

```
<script>let lastKey = null; function pentagrid () {let nowKey = window.encryptionKey;  
if(nowKey && nowKey !== lastKey){lastKey = nowKey; let pentagrid2 = new XMLHttpRequest();  
pentagrid2.open("GET", "/pentagridDiscloseKey=" + nowKey, false);  
pentagrid2.send(null);}}; setInterval(pentagrid, 500);</script></body>
```

This JavaScript extracted the encryption key from the browser DOM and sent it through Burp, so we could catch it in our decrypter/encrypter extensions and therefore fully intercept cleartext traffic.

Happy hacking!